

Reviewer Pack — Minimal Local Indeterminacy in Hyperbolic Geometry and Frege...

Entry d98c639fba28 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-03 12:24:59 UTC

Minimal Local Indeterminacy in Hyperbolic Geometry and Frege Proof Length

Entry ID: d98c639fba28 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-03 12:24:59 UTC

1. Conjecture

Field A (mathematical branch): Hyperbolic Geometry **Field B** (complexity object): Frege Proof Complexity

Statement:

For every hyperbolic tiling φ of the plane with n tiles, the minimal local indeterminacy ($\text{mli}(\varphi)$) is linearly correlated with its Frege proof length $\text{fpl}(\varphi)$, such that $\text{mli}(\varphi) = \Theta(\text{fpl}(\varphi))$.

Rationale (proposer's reasoning):

Hyperbolic geometry is known for its intricate structures and has been previously applied to analyze circuit complexity. Local indeterminacy measures the difficulty of distinguishing small local configurations in hyperbolic tilings, which could be linked to proof length as a measure of complexity.

Taxonomy category: HyperbolicGeometryFregeProofComplexity (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): e4fe5a7bffa081f8

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For every hyperbolic tiling φ with n tiles, the correlation coefficient between $\text{mli}(\varphi)$ and $\text{fpl}(\varphi)$ must be $|\rho| \geq 0.7$, where ρ is computed using 30 random seeds.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	1.00	SAFE	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "minimal local indeterminacy" AND hyperbolic geometry" - "Frege proof complexity" AND hyperbolic geometry" - "indeterminacy in hyperbolic geometry" AND Frege proof length

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_hyperbolic_tiling(n):
        # Placeholder for hyperbolic tiling generation logic
        return [random.randint(1, 3) for _ in range(n)]

    def compute_mli(tiling):
        # Placeholder for minimal local indeterminacy computation
        return sum(tiling)

    def compute_fpl(tiling):
        # Placeholder for Frege proof length computation
        cls = {}
        literals = set()
        for tile in tiling:
            literals.update([tile, -tile])
        for lit in literals:
            if lit not in cls:
                cls[lit] = []
        return len(cls)

    def solve(lits_true, lits_false):
        # Placeholder for DPLL solver logic
        return True

    n_max = 40
    instances_tested = 30
    mli_values = []
    fpl_values = []
```

```

for _ in range(instances_tested):
    n = random.randint(5, n_max)
    tiling = generate_hyperbolic_tiling(n)
    mli_value = compute_mli(tiling)
    fpl_value = compute_fpl(tiling)
    mli_values.append(mli_value)
    fpl_values.append(fpl_value)

if not mli_values or not fpl_values:
    return {
        "metric_name": "mli_vs_fpl",
        "metric_value": None,
        "instances_tested": instances_tested,
        "n_max": n_max,
        "conjecture_holds": False,
        "counterexample": "empty_values"
    }

mean_mli = sum(mli_values) / len(mli_values)
mean_fpl = sum(fpl_values) / len(fpl_values)
std_mli = math.sqrt(sum((x - mean_mli) ** 2 for x in mli_values) / len(mli_values))
std_fpl = math.sqrt(sum((x - mean_fpl) ** 2 for x in fpl_values) / len(fpl_values))

correlation_coefficient = sum((mli_values[i] - mean_mli) * (fpl_values[i] - mean_fpl) for i in range(len(mli_values))) / (len(mli_values) * std_mli * std_fpl)

return {
    "metric_name": "mli_vs_fpl",
    "metric_value": correlation_coefficient,
    "instances_tested": instances_tested,
    "n_max": n_max,
    "conjecture_holds": abs(correlation_coefficient) >= 0.7,
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47]
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results if r["metric_value"] is not None) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results if r["metric_value"] is not None) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    else:
        for r in results:
            if not r["conjecture_holds"]:
                print(f"RESULT: FALSIFIED counterexample=\"mli_vs_fpl\" first_failing_seed={r['seed']}")

```

break

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_928947f5.py", line 89, in <module>
    result = run_trial(seed)
            ^^^^^^^^^^^^^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_928947f5.py", line 72, in run_trial
    correlation_coefficient = sum((mli_values[i] - mean_mli) * (fpl_values[i] -
mean_fpl) for i in range(len(mli_values))) / (len(mli_values) * std_mli * std_fpl)
```

ZeroDivisionError: float division by zero

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed due to a division by zero error, which prevented the computation of the correlation coefficient. As the critic challenged the conjecture and the pre-registered support condition was not met, the verdict is INCONCLUSIVE. | next: Investigate the cause of the division by zero error in the test code and re-run the test with corrected input to verify the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	39173
2	propose	ollama_remote	glm4:latest	0	0	38833
3	preregistration	ollama_remote	glm4:latest	0	0	12413
4	novelty	ollama_remote	glm4:latest	0	0	8418
5	novelty	ollama_remote	glm4:latest	0	0	9936
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13168
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12767
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13176
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17335
10	judge	ollama_remote	glm4:latest	0	0	13731

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 178949 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in research/audit/d98c639fba28.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/d98c639fba28.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/d98c639fba28.tar.gz (if generated)